

מערכות הפעלה – מטלה 3

ת"ז – 212214316 , 211544960

TQ1

1. כאשר המערכת מגיעה למצב steady Thread A , ימצא ב-Q2 ו-Thread B ימצא ב-Q0.
Thread A – מכיוון שהוא מתחיל ב-Q0 ורץ 10 ms ברצף, הוא יעבור ל-Q1, שם ירוץ 20 ms ברצף, וירד ל-Q2, שם יישאר לשארית הריצה שלו, לפי ההגדרה ש-thread שנמצא ב-Q2 יישאר ב-Q2.
Thread B – יתחיל ב-Q0 וקורא ל-yield() לאחר 9.9 שניות, לכן יישאר ב-Q0 מכיוון שלא רץ לאורך כל ה-quantum.
2. Thread A ירוץ 1 full quanta לפני ש-Thread B משתלט על ה-CPU.
בתחילת הריצה Thread A ירוץ ויסיים את ה-quantum ב-Q0, וירד ל-Q1. לאחר מכן Thread B יתחיל לרוץ ב-Q0, ולאחר 9.9 מילי-שניות הוא יקרא ל-yield().
בסיום ה-yield הוא ירד לסוף Q0 אך עדיין יהיה מתועדף על Thread A שנמצא ב-Q1. לכן תתבצע ריצה של 1 full quanta.
נשים לב כי בסיום ה-yield ולפי סעיף 3 – Thread B יישאר ב-Q0 לכן ירוץ שוב 9.9 מילי-שניות, ושוב לאחר קריאת yield יישאר ב-Q0, וכך יישאר בלולאה בה Thread A לא ירוץ לעולם. מצב זה מהווה ה-steady state, ובו Thread B יקבל את כל זמן ה-CPU מכך ש-Thread A יקבל 0% מזמן ה-CPU.
3. גרסה זו של ה-MLFQ מניחה בטעות ש-threads שקוראים ל-yield לפני סיום ה-quantum שלהם, הם תהליכי I/O שמוותרים על ה-CPU כדי לחכות לאירוע חיצוני כלשהו, ולא תהליכי CPU-bound שמרמים באופן מכוון כדי לשמור על תיעדוף גבוה.
4. Thread B לעולם לא ירד ל-Q1 מכיוון שיבצע yield() לפני שיסיים את ה-quantum שלו, לכן לפי כלל 3 לא ירד לעולם מ-Q0.

TQ2

1. A page fault occurs when the accessed page's PTE has valid bit = 0.
The CPU then traps to the OS, which loads the page into a **physical frame**.
- 2.

Access?	1	2	3	4	1	2	5	1	2	3	4	5
Fault?	F	F	F	F	F	F	F	H	H	F	F	F
Resident pages	{1}	{1,2}	{1,2,3}	{2,3,4}	{3,4,1}	{4,1,2}	{1,2,5}	{1,2,5}	{1,2,5}	{1,2,3}	{2,3,4}	{3,4,5}

Total LRU page faults: 10.

3.

Access	F/H	Frame 0	Frame 1	Frame 2	Hand
1	F	(1,1)	-	-	1
2	F	(1,1)	(2,1)	-	2
3	F	(1,1)	(2,1)	(3,1)	0
4	F	(4,1)	(2,0)	(3,0)	1
1	F	(4,1)	(1,1)	(3,0)	2
2	F	(4,1)	(1,1)	(2,1)	0
5	F	(5,1)	(1,0)	(2,0)	1
1	H	(5,1)	(1,1)	(2,0)	1
2	H	(5,1)	(1,1)	(2,1)	1
3	F	(5,0)	(3,1)	(2,0)	2
4	F	(5,0)	(3,1)	(4,1)	0
5	H	(5,1)	(3,1)	(4,1)	0

Total Clock page faults: 9.

Exact LRU produced 10 faults, Clock produced 9 faults. 4.

הטענה לא נכונה – אלגוריתם Clock יכול לייצר פחות page faults מאשר exact LRU, מכיוון ש-LRU מקבל החלטות פינוי אך ורק על סמך היסטוריית העבר ולא חוזה בצורה מושלמת את הגישות העתידיות של התוכנית – ניתן לראות בדוגמה בשאלה זו.

5. דף מלוכלך – זהו דף שהשתנה בזיכרון מאז שנטען, ולכן מערכת ההפעלה חייבת לכתוב את השינויים האלו חזרה לדיסק (swap), כדי לשמור על עקביות הנתונים לפני שניתן יהיה לעשות שימוש חוזר, בעוד ש-clean page זהה בלעותק שנמצא בדיסק וניתן פשוט לדרוס אותו.

TQ3

1. Command A copies 0 bytes of the file's content. Command B copies 5 GiB / 5,368,709,120 bytes.

מה שפקודה A בפועל משנה בדיסק, זה את שם הנתיב בתוך אותה מערכת הקבצים, בלי לשנות את ה-inode או את בלוקי המידע של הקובץ.

2. After Command B removes the entry /home/alice/report.pdf, 1 hard links still point to that /home inode.

Running cat /home/alice/backup/report.pdf then shows original content. הסבר – לאחר הפעלת פקודה B – ה-link - /home/alice/report.pdf איננו מצביע יותר לקובץ, אך עדיין יש לנו את הלינק /home/alice/backup/report.pdf, לכן יש לנו link אחד שעדיין מפנה ל/home inode. כמו כן יוצג התוכן המקורי מכיוון שעדיין יש לנו לינק לקובץ כי ה-hard link הזה עדיין מצביע ל-inode המקורי, וה-inode המקורי לא נמחק כל עוד יש אליו לפחות hard link אחד.

3. False – hard link לא יכול להפנות ל-inode בחלק אחר, מכיוון שיש לכל חלק inode table ומערכת קבצים שונה. מסיבה זו הפקודה בשאלה תיכשל – מנסים לקשר קבצים בחלקים אחרים – דבר לא אפשרי.

4. **Error (broken link)**

Symbolic link שומר את הנתבי בתור טקסט ולא בתור קובץ, ולכן הפקודה בשאלה מנסה לכתוב נתיב שכבר לא קיים (מכיוון שביצוע פקודת B משנה את הכתובת), ולכן אנחנו נתקלים בתקלה.

$$5.1. \frac{4096}{4} = 1,024$$

$$5.2. \frac{5 \cdot 2^{30}}{2^{12}} = 1,310,720$$

$$3. 12 + 1,024 + 1,024 \cdot 1,024 = 12 + 1,024 + 1,048,576 = 1,049,612$$

4. כן – מכיוון שגודל הקובץ הוא 5 GiB, כלומר 1,310,720 בלוקים – וההגבלה של שימוש עד (וכלל) של ה-double indirect הוא 1,049,612 – נראה שנצטרך להשתמש בבלוקים של ההפרש ביניהם שהם בשימוש של ה-triple indirect.